

1. LINEAR APPROXIMATION TABLE FOR THE S-BOX

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
8	4	2	1	C	6	3	D	A	5	E	7	F	B	9	0

Algorithm

Creating a table with the binary representation of each number.
i.e. `input_bits[16][4]`

For each entry in the table we check for '1's in the binary representation of the input sum and the output sum. If it is equal to '1' then we XOR that bit with a temp variable for each (input and output) (temp initialized to 0).

We check if the XOR of the input bit positions decided by the row and the output bit positions decided by the column is equal. If it is equal we increment the counter.

We subtract 8 from the final value of the counter and save the value in the table for the corresponding row and column.

CODE:

```

#include<iostream>
#include<iomanip>

using namespace std;

int bit(int num, int pos)
{
    return (num>>pos)%2;
}

int main()
{
    int
input[]={0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
    int
output[]={8,4,2,1,12,6,3,13,10,5,14,7,15,11,9,0};

    int input_bits[16][4]={};
    int output_bits[16][4]={};
    int table[16][16]={};
    int counter=0;
    int tempi=0,tempo=0;

    for(int i=0;i<16;i++)
    {
        for(int j=0;j<4;j++)
        {
            input_bits[i][j]=bit(input[i],j);
            output_bits[i][j]=bit(output[i],j);
            //cout<<input_bits[i][j]<<" ";

        }
        //cout<<endl;
    }
    // Going through each row and column of the table
    for(int row=0;row<16;row++) {
        for(int col=0;col<16;col++) {
            counter=0;
            // Looping through each bit
            for(int i=0;i<16;i++) {
                tempi=0,tempo=0;

                for(int j=0;j<4;j++) {
                    // If the bit position contains 1 then XOR
                    if(input_bits[row][j]==1) {
                        tempi=input_bits[i][j]^tempi;
                    }

                    if(input_bits[col][j]==1) {
                        tempo=output_bits[i][j]^tempo;
                    }
                }
                // Checking if the L.H.S. and R.H.S are
                // equal if yes then increment counter
                if(tempo==tempi) {
                    counter++;
                }
            }
            table[row][col]=counter-8;
        }
    }

    // Printing the formatted output.

    cout<<" ";
    for(int i=0;i<16;i++){
        cout<<setw(2)<<i<<" ";
    }
    cout<<endl;
    cout<<setw(32)<<"-----"
    -----";
    cout<<endl;

    for(int i=0;i<16;i++) {
        cout<<setw(2)<<i<<" | ";

        for(int j=0;j<16;j++) {
            cout<<setw(2)<<table[i][j]<<" ";
        }
        cout<<endl;
    }
}

```

OUTPUT:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

```
0 | 8 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 | 0 2 -2 0 2 0 0 -2 -4 -2 -2 0 2 0 -4 2
2 | 0 2 0 2 -2 0 -2 0 -2 0 2 -4 -4 -2 0 2
3 | 0 0 2 2 0 4 2 -2 -2 -2 0 0 2 -2 4 0
4 | 0 2 0 -2 0 2 0 -2 2 -4 2 0 -2 0 -2 -4
5 | 0 4 -2 -2 2 2 0 4 -2 2 0 0 0 0 2 -2
6 | 0 0 4 0 2 2 -2 2 0 0 0 4 -2 -2 -2 2
7 | 0 -2 -2 0 4 -2 2 0 0 -2 -2 0 -4 -2 2 0
8 | 0 2 2 0 0 -2 -2 0 2 0 -4 -2 2 -4 0 -2
9 | 0 0 0 4 2 2 -2 2 2 -2 -2 -2 0 4 0 0
10 | 0 4 2 2 -2 -2 4 0 0 0 -2 2 -2 2 0 0
11 | 0 -2 4 -2 0 -2 0 2 -4 -2 0 -2 0 2 0 -2
12 | 0 0 2 2 4 0 2 -2 0 4 2 -2 0 0 -2 -2
13 | 0 -2 0 -2 -2 4 2 0 0 2 -4 -2 -2 0 -2 0
14 | 0 -2 -2 4 -2 0 0 2 -2 0 0 2 0 -2 -2 -4
15 | 0 0 0 0 0 0 4 4 2 -2 2 -2 2 -2 -2 2
```

Process returned 0 (0x0) execution time : 0.041 s

Press any key to continue.

1. Look at the SPN from Figure 1 from Heys's tutorial. Use the linear approximation table from the previous problem to find a linear approximation using 3 active S-boxes, and use the piling-up lemma to estimate the bias of

$$P_{16} \text{ XOR } U_{4,1} \text{ XOR } U_{4,9} = 0.$$

Note: There is more than one set of 3 active S-boxes that will work. Use the one that gives you the largest absolute bias.

Explain your answer at the level of detail of slides.

To obtain the bias of $P_{16} \text{ XOR } U_{4,1} \text{ XOR } U_{4,9} = 0$, we proceeded to do the following steps:

- First, we looked at the S-Boxes figure similar to the one provided below to have a visual idea of how to figure out the S-Boxes that would make possible a linear expression. We traced all given input in order to know the values that we are working with to come up with a test that yields the bias that best fits the above expression.
- Second, tracing all S-Boxes from round 1, as shown in page 2 right after the figure provided, and tested each possible condition to figure out which S-Box has the highest bias.
- Third, a similar approach to the second step was taken in round 3 to figure out which S-Box has the highest bias, and compare this with the results obtained in round 1.

Finally, results in round one gave us S2, 1 as the box with the highest bias results in round 3 also gave us S2,1 as the box with the highest bias. Therefore, we were convinced that the result obtained was the right result and therefore proceeded to calculate the entire bias. The total bias summed up to be Bias = 5.098.

Diagram for P_{16} XOR $U_{4,1}$ XOR $U_{4,9}$ Bias Calculation

The diagram illustrates the bias calculation for P_{16} XOR $U_{4,1}$ XOR $U_{4,9}$ across four rounds. The components and their connections are as follows:

- ROUND 1:** Block $S_{1,4}$ (red) receives input P_{16} and outputs $V_{1,16}$, $U_{2,16}$, $V_{1,15}$, and $U_{2,12}$.
- ROUND 2:** Blocks $S_{2,1}$ (red), $S_{2,2}$ (blue), $S_{2,3}$ (blue), and $S_{2,4}$ (blue) receive inputs from Round 1 and each other. $S_{2,1}$ outputs $V_{2,1}$, $U_{3,1}$, $U_{3,2}$, and $U_{3,3}$. $S_{2,2}$ outputs $V_{2,5}$ and $U_{3,4}$. $S_{2,3}$ outputs $V_{2,19}$ and $U_{3,3}$. $S_{2,4}$ outputs $V_{2,13}$ and $U_{3,3}$.
- ROUND 3:** Block $S_{3,1}$ (red) receives inputs from Round 2 and outputs $V_{3,1}$ and $U_{4,1}$.
- ROUND 4:** Blocks $S_{4,1}$ (red) and $S_{4,3}$ (red) receive inputs from Round 3 and output $U_{4,1}$ and $U_{4,19}$ respectively.

Calculations from the Look-Up Table – S-BOXES TABLE

Round One:

P16 = 0000 0000 0000 0001

1) P16 & V1, 13

V1, 13 = 0000 0000 0000 1000

Bias = $-1/4$

2) P16 & V1, 14

V1, 13 = 0000 0000 0000 0100

Bias = $1/8$

3) P16 & V1, 15

V1, 13 = 0000 0000 0000 0010

Bias = $-1/8$

4) P16 & V1, 16

V1, 16 = 0000 0000 0000 0001

Bias = $1/8$

S2, 1

Bias = $-1/4$

Round Three:

V3, 1 = 1000 0000 0000 0000

1) U1, 13 & V3, 1

U1, 13 = 0000 0000 0000 1000

Bias = $1/8$

2) U1, 14 & V3, 1

U1, 14 = 0000 0000 0000 0100

Bias = $1/8$

3) U1, 15 & V3, 1

U1, 15 = 0000 0000 0000 0010

Bias = $-1/8$

4) U1, 16 & V3, 1

U1, 16 = 0000 0000 0000 0001

Bias = $-1/4$

S2, 1

Bias = $-1/4$

Round Two:

U2, (13, 14, 15, 16) & V2, (1, 5, 9, 13) -> Bias = $-1/4$

S1, 4 = $-1/4$

S3, 1 = $-1/4$

S2, 1 = $-1/4$

S4, 1 or S4, 3 = $5/8$

Bias = $(1/4 * 1.4 * 1/4 * 5/8) + 1/2 = 0.5098$

Bias approximation = 0.5

Use this linear approximation to implement a linear cryptanalysis attack on this SPN:

- Randomly generate the round keys (80 bits total).
- Randomly generate $8 * \epsilon^{-2}$ different 16-bit plaintexts, and compute the corresponding ciphertexts, where ϵ is the bias of $P_{16} \text{ XOR } U_{4,1} \text{ XOR } U_{4,9} = 0$.
- Perform the attack that is described in the tutorial and the slides to compute 8 bits of the round key for round 5.

This part of the assignment was constructed in parts in order to achieve the closest result to each one of the steps told to do. These steps were developed as follows:

First, we proceeded to create a code that generates random keys of 80 bits length. The way that these keys were generated was by creating list of all possible input that can be used in a key, inputs from 0 – F. Then, since we are dealing with a 4 X 4 S-Box system, the keys were generated one at time, and each one of the key has a length of 20 bits. This way we were able to generate a key with a total length of 80 bits since there are total 4 sub-keys whose length sum up to be 80 bits. A list of the obtained keys was saved in order to avoid having any sub-key repeated.

Second, we proceeded to create a code that generates random inputs by implementing $8 * \epsilon^{-2}$, where ϵ is 0.5 the bias approximation obtained from the previous question in which $P_{16} \text{ XOR } U_{4,1} \text{ XOR } U_{4,9} = 0$. Inputs produced by these are of four bits size.

Third, we proceeded to encrypt the random obtained plaintext with the random generated key. The way this was done was by following the instruction of the tutorial to, as told, and using java properties to speed up and reduce coding while obtaining a more accurate result. The source of the random key, plaintext, and decryption of both are illustrated below. The first code is the code to generate random keys, second code is to generate random plaintext, and the third code is the main where both algorithm are called and mixed in order to encrypt.

Finally, we created another source code in which we proceed to do the linear attack. The way this was first achieved was by creating a look-up table obtained from the first question. Then, a list of possible sub-keys was created which range from 0 – F. The first sub-key of the list was mixed with all key of another list, a copy of the list, and this was looked up in the S-Box table fifteen times for each one of the fifteen keys. As a result, we obtained a bias table that reveals the bias of the possible keys of the system. Note that we were not able to find 8 bits of the fifth round because we ran out of time. However, you can see that all work needed to find such bits was done, and therefore, all we needed was to implement our attack algorithm such that we can make a smart guesses that lead us to the find 8 bits of the round 5. In conclusion, it is good to highlight that having key length of 80 bits makes the cipher easier to attack since is a very weak key length. In order for an AES encryption to be more difficult to break the key length must be greater or equal 128 bits, for more details about the work that we were not able to finish look at the source in the test1.java file.

Random Key Generator – Source Code

```
public class KeyRandomGen implements Comparable<char[]>{
```

```
    ArrayList<Character> list = new ArrayList<Character>();
    static ArrayList<char[]> keyList = new ArrayList<char[]>();
    static int size;
    private char[] key;
```

```
    static int kSize;
```

```
    public static void keys(char[] key){
        if(!keyList.contains(key))
            keyList.add(key);
        else
            System.out.println("checkinh");
    }
```

```
    public KeyRandomGen(){
```

```
        char[] temp = {'A', 'B', 'C', 'D', 'E', 'F'};
```

```
        int i = -1;
        int x = -1;
        while(++i != 16){
            String tmp = ""+i;
            if(i<10)
                list.add(tmp.charAt(0));
            else{
                list.add(temp[++x]);
            }
        }
```

```
    public static void main(String[] args){
        KeyRandomGen keyG = new KeyRandomGen();
        int keySize = 80;
        size = keySize/4;
        try {
            keyG.generateKeySet(size);
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        }
    }
```

```
    public void generateKeySet(int size) throws
FileNotFoundException, UnsupportedEncodingException{
        KeyRandomGen keyG = new KeyRandomGen();
        keyG.key = new char[size];
```

```
        PrintWriter writer = new PrintWriter("key.txt", "UTF-8");
        int i = 0;
        do{
            keys(keyG.generateKeys(size));

        } while( i++ != 3);
        String[] array = keyG.toString().split(",");
        System.out.println();
```

```
        for(int x=0; x< array.length; x++){
            writer.println(array[x]);
            System.out.println(array[x]);
        }
        System.out.println("\nSet of Keys : \n"+keyG);
        writer.close();
    }
```

```
    public void morekeys(){
```

```
        int i = 0;
        do{
            keys(generateKeys(size));
        } while( i++ != 3);
    }
```

```
    public char[] generateKeys(int size) {
        KeyRandomGen keyG = new KeyRandomGen();
        keyG.key = new char[size];
```

```
        int x = -1;
        while( ++x < size){

            int tmpK = new Random().nextInt(15);
            keyG.key[x] = keyG.list.get(tmpK);
        }
```

```
        if(KeyRandomGen.keyList.contains(keyG.key)){
```

```
            System.out.println("HERE");
            keyG.generateKeys(size);
        }
```

```
        System.out.print(keyList.size() + " Random Key = ");
        System.out.println(keyG.key);
```

```
        return keyG.key;
```

```
    public int compareTo(char[] o) {
```

```
        if( this.equals(o))
            return 0;
```

```
        else
            return -1;
```

```
    public String toString(){
```

```
        String temp = "";
        for( char[] cl: keyList){
```

```
            for(Character c: cl)
                temp += c;
```

```
            temp += ",";
```

```
        }
        return temp+"";
```

```
    }
```

```
}
```


Plaintext Generator – Source Code

```

public class InputRandom implements Comparable<char[]>{

    double bias = 0.509;

    ArrayList<Character> list = new ArrayList<Character>();
    static ArrayList<char[]> keyList = new ArrayList<char[]>();
    static int size;
    private char[] key;

    static int kSize;

    public static void keys(char[] key){
        if(!keyList.contains(key))
            keyList.add(key);
        else
            System.out.println("checkinh");
    }

    public InputRandom(){

        char[] temp = {'A', 'B', 'C', 'D', 'E', 'F'};

        int i = -1;
        int x = -1;
        while(++i != 16){
            String tmp = ""+i;

            if(i<10)
                list.add(tmp.charAt(0));
            else{
                list.add(temp[++x]);
            }
        }

    }

    public static void main(String[] args){

        InputRandom keyG = new InputRandom();

        int keySize = (int) (8*(1/(0.5*0.5)));
        size = keySize/4;
        try {
            keyG.generateKeySet(size);
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        }

    }

    public void generateKeySet(int size) throws
    FileNotFoundException, UnsupportedEncodingException{
        InputRandom keyG = new InputRandom();
        keyG.key = new char[size];

        int i = 0;
        do{
            keys(keyG.generateKeys(size));
        } while( i++ != 3);
        PrintWriter writer = new PrintWriter("input.txt", "UTF-8");
        String[] array = keyG.toString().split(",");
        System.out.println();
        for(int x=0; x< array.length; x++){
            writer.println(array[x]);
            System.out.println(array[x]);
        }
        System.out.println("\nSet of Inputs : \n"+keyG);
        writer.close();
    }

    public void morekeys(){
        int i = 0;
        do{
            keys(generateKeys(size));
        } while( i++ != 3);
    }

    public char[] generateKeys(int size) {
        InputRandom keyG = new InputRandom();
        keyG.key = new char[size];
        int x = -1;
        while( ++x < size){
            int tmpK = new Random().nextInt(15);
            keyG.key[x] = keyG.list.get(tmpK);
        }
        if(InputRandom.keyList.contains(keyG.key)){
            System.out.println("HERE");
            keyG.generateKeys(size);
        }
        System.out.print(keyList.size() + " Random Input = ");
        System.out.println(keyG.key);
        return keyG.key;
    }

    public int compareTo(char[] o) {

        if( this.equals(o))
            return 0;
        else
            return -1;
    }

    public String toString(){
        String temp = "";
        for( char[] cl: keyList){
            for(Character c: cl)
                temp += c;
            temp+=",";
        }
        return temp+"";
    }
}

```

Encryption Source Code

```
public class SPN_test {
    public static void main(String[] args) {

        InputRandom inRan = new InputRandom();

        int keySize = (int) (8*(1/(0.5*0.5)));
        int size = keySize/4;
        try {
            inRan.generateKeySet(size);
        } catch (FileNotFoundException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        } catch (UnsupportedEncodingException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

        KeyRandomGen keyG = new KeyRandomGen();
        int keySize2 = 80;
        size = keySize2/4;
        try {
            keyG.generateKeySet(size);
        } catch (FileNotFoundException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        } catch (UnsupportedEncodingException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

        System.out.println();

        GetBytes getInput = new GetBytes("input.txt", 16);
        byte[] in = getInput.getBytes();
        GetBytes getKey = new GetBytes("key.txt", 80);
        byte[] key = getKey.getBytes();
        SPNencrypt SPN = new SPNencrypt(key, 4);
        Print.printArray("Plaintext: ", in);
        Print.printArray("Key: ", key);
        byte[] out = new byte[16];
        SPN.Cipher(in, out);
        Print.printArray("Ciphertext: ", out);
    }
}
```

Generated Output – Run InputRandom.java and KeyRandomGen.java for more details.

0 Random Input = 49CB133D

1 Random Input = 2D198E1C

2 Random Input = 4BB28D08

3 Random Input = 853DD614

49CB133D

2D198E1C

4BB28D08

853DD614

Set of Inputs :

49CB133D,2D198E1C,4BB28D08,853DD614,

0 Random Key = A9BE8493A1D0332BC204

1 Random Key = 4C1E1514C6D2717B574E

2 Random Key = 798B64CA9621815DA9EC

3 Random Key = DB4951EA47D4746A591D

A9BE8493A1D0332BC204

4C1E1514C6D2717B574E

798B64CA9621815DA9EC

DB4951EA47D4746A591D

Set of Keys :

A9BE8493A1D0332BC204,4C1E1514C6D2717B574E,798B64CA9621815DA9EC,DB4951EA47D4746A591D,

Plaintext: 49 cb 13 3d 02 d1 98 e1 80 4b b2 8d 08 08 53 dd

Key: a9 be 84 93 a1 d0 33 2b c2 04 04 c1 e1 51 4c 6d 27 17 b5 74 a0

79 8b 64 ca 96 21 81 5d a9 ec 0d b4 95 1e a4 7d 47 46 a5 91 90 c0 c0

c0 c0

c0 c0 c0 c0 c0 c0 c0 c0 c0 c0 c0 c0 c0 c0 c0

Start round 1: e0 75 97 ae a3 01 ab ca 42 4f b6 4c e9 59 1f b0

Start round 2: 8d bb 8b b4 7f e1 a9 c9 f8 15 ff 5e 30 a2 ff 63

Start round 3: f6 ce e9 80 a5 29 d6 07 5c 12 0f c0 2d 52 0a cc

Start round 4: 82 b3 9b da 43 64 fd 58 9a 9f e0 89 3f f3 02 f0

Start round 5: 17 df ca 95 5e 08 aa 50 eb b7 9d ba 19 2a 8f c6

Start round 6: d5 85 e1 be a1 9c 4f b2 9b 86 89 67 a4 47 5b e3

Start round 7: 04 f8 8f b4 5a d2 ff 52 38 97 8c 44 e3 3d 86 d2

Start round 8: 8a bb 54 f2 28 58 3f 75 35 8c 41 72 1f b1 09 76

Start round 9: 62 47 68 49 d2 91 f6 03 76 57 01 74 c7 64 fb 7c

Start round 10: b6 63 cd 43 3b 8a b7 ae a7 1e d5 ec dc 69 1d e0

Ciphertext: 95 3a 7d 4a aa 95 d0 f9 38 0a 60 53 82 a8 26 15

Look-Up and Bias Table for Linear Attack

S-Boxes or Look-Up Table

8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	2	-2	0	2	0	0	-2	-4	-2	-2	0	2	0	-4	2
0	2	0	2	-2	0	-2	0	-2	0	2	-4	-4	-2	0	2
0	0	2	2	0	4	2	-2	-2	-2	0	0	2	-2	4	0
0	2	0	-2	0	2	0	-2	2	-4	2	0	-2	0	-2	-4
0	4	-2	-2	2	2	0	4	-2	2	0	0	0	0	2	-2
0	0	4	0	2	2	-2	2	0	0	0	4	-2	-2	-2	2
0	-2	-2	0	4	-2	2	0	0	-2	-2	0	-4	-2	2	0
0	2	2	0	0	-2	-2	0	2	0	-4	-2	2	-4	0	-2
0	0	0	4	2	2	-2	2	2	-2	-2	-2	0	4	0	0
0	4	2	2	-2	-2	4	0	0	0	-2	2	-2	2	0	0
0	-2	4	-2	0	-2	0	2	-4	-2	0	-2	0	2	0	-2
0	0	2	2	4	0	2	-2	0	4	2	-2	0	0	-2	-2
0	-2	0	-2	-2	4	2	0	0	2	-4	-2	-2	0	-2	0
0	-2	-2	4	-2	0	0	2	-2	0	0	2	0	-2	-2	-4
0	0	0	0	0	0	4	4	2	-2	2	-2	2	-2	-2	2

Bias Table

00	0.5	01	0	02	0	03	0	04	0	05	0	06	0	07	0	08	0	09	0	0A	0	0B	0	0C
10	0	11	0.125	13	0	14	0.125	15	0	16	0	18	-0.25	1B	0	1C	0.125	1D	0	1E	-0.25	1F	0.125	
20	0	21	0.125	22	0	23	0.125	25	0	27	0	29	0	2A	0.125	2B	-0.25	2C	-0.25	2E	0	2F	0.125	
30	0	31	0	32	0.125	33	0.125	34	0	35	0.25	36	0.125	3A	0	3B	0	3C	0.125	3E	0.25	3F	0	
40	0	41	0.125	42	0	44	0	45	0.125	46	0	48	0.125	49	-0.25	4A	0.125	4B	0	4D	0	4F	-0.25	
50	0	51	0.25	54	0.125	55	0.125	56	0	57	0.25	59	0.125	5A	0	5B	0	5C	0	5D	0	5E	0.125	
60	0	61	0	62	0.25	63	0	64	0.125	65	0.125	67	0.125	68	0	69	0	6A	0	6B	0.25	6F	0.125	
70	0	73	0	74	0.25	76	0.125	77	0	78	0	7B	0	7C	-0.25	7E	0.125	7F	0					
80	0	81	0.125	82	0.125	83	0	84	0	87	0	88	0.125	89	0	8A	-0.25	8C	0.125	8D	-0.25	8E	0	
90	0	91	0	92	0	93	0.25	94	0.125	95	0.125	97	0.125	98	0.125	9C	0	9D	0.25	9E	0	9F	0	
A0	0	A1	0.25	A2	0.125	A3	0.125	A6	0.25	A7	0	A8	0	A9	0	AB	0.125	AD	0.125	AE	0	AF	0	
B0	0	B2	0.25	B4	0	B6	0	B7	0.125	B8	-0.25	BA	0	BC	0	BD	0.125	BE	0					
C0	0	C1	0	C2	0.125	C3	0.125	C4	0.25	C5	0	C6	0.125	C8	0	C9	0.25	CA	0.125	CC	0	CD	0	
D0	0	D2	0	D5	0.25	D6	0.125	D7	0	D8	0	D9	0.125	DA	-0.25	DD	0	DF	0					
E0	0	E3	0.25	E5	0	E6	0	E7	0.125	E9	0	EA	0	EB	0.125	EC	0	EF	-0.25					
F0	0	F1	0	F2	0	F3	0	F4	0	F5	0	F6	0.25	F7	0.25	F8	0.125	FA	0.125	FC	0.125	FF	0.125	

Generated Output – Run LinearAttack.java and or see LinearAttackO.txt for more details.

Better View For the Bias Table

Table:

00	0.5	01	0	56	0	57	0.25	B4	0	B6	0
02	0	03	0	59	0.125	5A	0	B7	0.125	B8	-0.25
04	0	05	0	5B	0	5C	0	BA	0	BC	0
06	0	07	0	5D	0	5E	0.125	BD	0.125	BE	0
08	0	09	0	60	0	61	0	C0	0	C1	0
0A	0	0B	0	62	0.25	63	0	C2	0.125	C3	0.125
0C	0	0D	0	64	0.125	65	0.125	C4	0.25	C5	0
0E	0	0F	0	67	0.125	68	0	C6	0.125	C8	0
10	0	11	0.125	69	0	6A	0	C9	0.25	CA	0.125
13	0	14	0.125	6B	0.25	6F	0.125	CC	0	CD	0
15	0	16	0	70	0	73	0	D0	0	D2	0
18	-0.25	1B	0	74	0.25	76	0.125	D5	0.25	D6	0.125
1C	0.125	1D	0	77	0	78	0	D7	0	D8	0
1E	-0.25	1F	0.125	7B	0	7C	-0.25	D9	0.125	DA	-0.25
20	0	21	0.125	7E	0.125	7F	0	DD	0	DF	0
22	0	23	0.125	80	0	81	0.125	E0	0	E3	0.25
25	0	27	0	82	0.125	83	0	E5	0	E6	0
29	0	2A	0.125	84	0	87	0	E7	0.125	E9	0
2B	-0.25	2C	-0.25	88	0.125	89	0	EA	0	EB	0.125
2E	0	2F	0.125	8A	-0.25	8C	0.125	EC	0	EF	-0.25
30	0	31	0	8D	-0.25	8E	0	F0	0		
32	0.125	33	0.125	90	0	91	0	F1	0		
34	0	35	0.25	92	0	93	0.25	F2	0		
36	0.125	3A	0	94	0.125	95	0.125	F3	0		
3B	0	3C	0.125	97	0.125	98	0.125	F4	0		
3E	0.25	3F	0	9C	0	9D	0.25	F5	0		
40	0	41	0.125	9E	0	9F	0	F6	0.25		
42	0	44	0	A0	0	A1	0.25	F7	0.25		
45	0.125	46	0	A2	0.125	A3	0.125	F8	0.125		
48	0.125	49	-0.25	A6	0.25	A7	0	FA	0.125		
4A	0.125	4B	0	A8	0	A9	0	FC	0.125		
4D	0	4F	-0.25	AB	0.125	AD	0.125	FF	0.125		
50	0	51	0.25	AE	0	AF	0				
54	0.125	55	0.125	B0	0	B2	0.25				